

SPOT: Getting Started

Using the SPOT consoles, setting up your computer,
and running simulations without the GUI

Spacecraft Proximity Operations Testbed
Spacecraft Robotics and Control Laboratory, Carleton University

This guide is `SPOT_Tutorial.pdf` in the project folder. It describes SPOT 5
with the SPOT Console (2026). Build it there with `latexmk -pdf SPOT_Tutorial.tex`.

Contents

1 What SPOT Is, and How the Pieces Fit **3**

2 Setting Up Your Computer **4**

2.1 What You Need . . . 4
2.2 Windows . . . 4
2.3 Linux . . . 4
2.4 A New Project . . . 5
2.5 First Start . . . 5

3 The Local Console (SPOT Local Simulations) **6**

3.1 Around the Page . . . 6
3.2 Simulation Tab . . . 7
3.3 Experiment Tab . . . 8
3.4 Results Tab . . . 10
3.5 Settings and Your Own Parameters . . . 10
3.6 Spacecraft Designer . . . 10

4 The Hardware Console (SPOT Hardware Interfaces) **11**

4.1 PhaseSpace Tab . . . 11
4.2 Interactive Mode . . . 12
4.3 Hardware Tab . . . 12
4.4 For Lab Administrators . . . 13
4.5 Calibration Tab . . . 13

5 Running Simulations Without the GUI **14**

5.1 One Simulation . . . 14
5.2 Looking at the Results . . . 15
5.3 From a Terminal (Batch) . . . 15
5.4 A Parameter Sweep . . . 15
5.5 Experiments from MATLAB . . . 16

6 The Arm: Servos, Friction and Calibration **16**

6.1 How the Arm Is Simulated . . . 16
6.2 The Arm's Controllers . . . 16
6.3 The Friction Model . . . 17
6.4 Calibrating the Friction in the Lab . . . 17

6.5 The Arm on the Hardware 18

7 Inside the Diagram: The Order of a Step **19**

7.1 Why the Order Matters in SPOT 19

7.2 Virtual and Atomic Subsystems 19

7.3 SPOT's Order 20

7.4 Demonstration 21

7.5 The Plant Loop, and Stores That Hold a Value 22

7.6 When You Change the Diagram 22

A Function Reference **23**

B The Console's Commands and the Non-MATLAB Parts **28**

1 What SPOT Is, and How the Pieces Fit

SPOT is the lab's testbed of three air-bearing spacecraft platforms, the **Chaser**, the **Target** and the **Obstacle** (the Chaser can carry a robotic arm), floating on a 3.5×2.4 m granite table. The consoles show them by these names and colours (Chaser silver, Target gold, Obstacle blue); the code, the diagram, the settings and the data files keep their older keys `RED`, `BLACK` and `BLUE` (and `ARM`), e.g. `RED_Px_m` is the Chaser's x position. A PhaseSpace motion-capture system tracks them. You design a guidance and control law in a Simulink diagram, simulate it on your own computer, then build it and run it on the platforms.

You work with SPOT through two web pages, called *consoles*:

The local console ("SPOT Local Simulations") runs on *your* computer and is started from MATLAB with the command `SPOT`. Its main tabs are **Simulation** (settings, simulation and 3D replay), **Experiment** (build, deploy and run on the platforms) and **Results** (load and plot data), with the **Designer** and **Calibration** (the project's copy of the lab's calibration). Everything it does is carried out by your MATLAB session.

The hardware console ("SPOT Hardware Interfaces") runs on the lab's *lab console*, a Jetson TX2 at `http://192.168.1.116:8080` (the Experiment tab's **Lab Console (TX2) Address**), reachable only on the lab network. Its tabs are **PhaseSpace** (the motion-capture stream and a live 3D view), **Hardware** (air-bearing pucks, thrusters and thruster checks on each platform), the **Designer** and **Calibration** (measuring the lab: Section 4.5). The local console's Experiment tab talks to it for you.

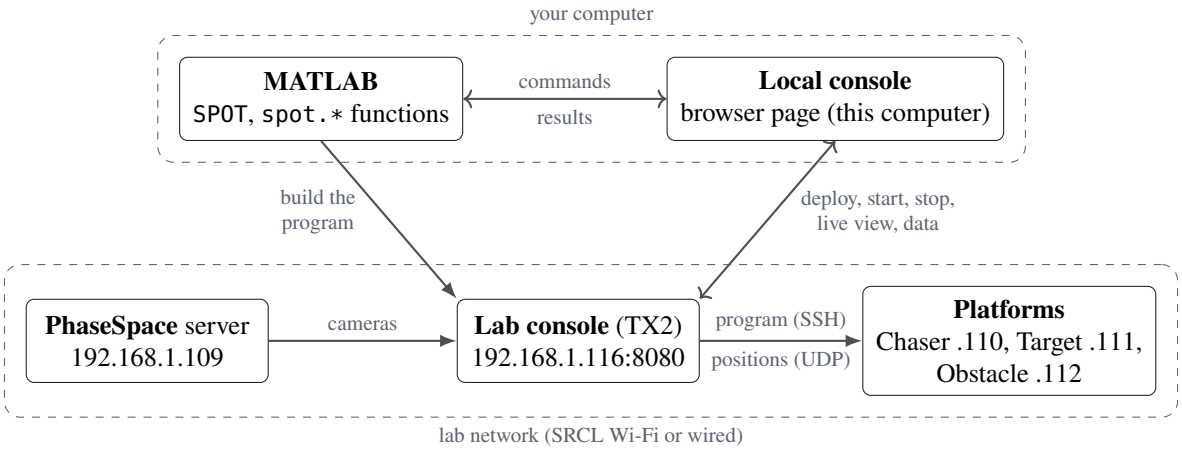


Figure 1: Your computer runs MATLAB and the local console. On the lab network, the lab console (TX2) builds your program, sends it to the platforms, streams PhaseSpace positions to them, and collects their data.

The Project Folder

Each project is a copy of the SPOT template folder, `template`, made with the MATLAB function `New_SPOT_Project` (Section 2.4), which puts it in `projects`. Its top level is kept short:

`SPOT.m`	Starts the local console (`SPOT`, `SPOT open`, `SPOT stop`).
`<name>.slx`	The Simulink diagram, named after the project (the template's `Template_v5_0_0_2026a_Jetson_PSClock.slx`); edit it for your own experiment.
`SPOT_User_Parameters.m`	Your own workspace variables; runs after the console's settings are applied.
`spot_settings.json`	The console's settings, saved automatically.

`+spot/`	All of SPOT's code, as MATLAB packages (Appendix A); `+spot/+console` also holds the consoles' web code.
`resources/`	The PhaseSpace marker calibration, the arm calibration script, the reference pictures the console shows.
`Saved Data/`	Experiment data and saved simulations (the console creates it when it first saves something).
`SPOT_Tutorial.pdf`	This guide.

2 Setting Up Your Computer

2.1 What You Need

MATLAB	**R2026a or newer** (the template diagram is saved in R2026a; older releases can't open it).
For simulations	Simulink, DSP System Toolbox, Control System Toolbox (for the LQR gains).
For experiments	Also MATLAB Coder, Simulink Coder, Embedded Coder, and the *MATLAB Coder Support Package for NVIDIA Jetson and NVIDIA DRIVE Platforms* (MATLAB: **Home** → **Add-Ons**).
Python	3.6 or newer, standard library only (nothing to `pip install`). It runs the local console's small web server. If it is missing, `SPOT` offers to install it (below).
Web browser	Any current browser (Chrome, Firefox, Edge, Brave, Safari).
For experiments	A connection to the lab network (SRCL Wi-Fi, or a cable in the lab), and the TX2's password from a lab administrator (needed once).

2.2 Windows

1. Install MATLAB R2026a with the toolboxes above (MathWorks installer, or **Add-Ons** afterwards).
2. Python: nothing to do beforehand. If `SPOT` can't find one, it says so and offers **Install Python**, which installs it for your user only (with `winget`, or python.org's installer when there is no `winget`; no administrator rights needed). To install it yourself: python.org, and tick **Add python.exe to PATH**. SPOT tries the Python MATLAB is set up to use (`pyenv`), then `py -3`, `python`, `python3`, then where the installers put it.
3. Make your project: run `New_SPOT_Project('MyProject')` in MATLAB, from the folder that holds `template` (Section 2.4). Avoid OneDrive-synced folders while building experiments, since syncing can lock files.
4. Start MATLAB, **cd** into the project folder, and type `SPOT` (Section 2.5).
5. If Windows Firewall asks whether Python may use the network, *private networks* is enough: the local console only listens on your own computer (127.0.0.1).

2.3 Linux

1. Install MATLAB R2026a with the toolboxes above.
2. Python 3 comes with most distributions. If it is missing, `SPOT` offers **Install Python**, which installs it with your package manager (apt, dnf, yum, zypper or pacman) and asks for your password in a system dialog; without a desktop it prints the command instead (e.g. `sudo apt install python3`).
3. The local console opens your default browser with `xdg-open` (part of every desktop install).

4. Make your project: run `New_SPOT_Project('MyProject')` in MATLAB, from the folder that holds `template` (Section 2.4).
5. Start MATLAB, **cd** into the project folder, and type `SPOT`.

2.4 A New Project

`New_SPOT_Project.m` sits next to the `template` and `projects` folders. Run it in MATLAB:

```
New_SPOT_Project('MyProject')              % creates projects/MyProject, with MyProject
    .slx
New_SPOT_Project                           % asks for the name
New_SPOT_Project('MyProject', 'D:\SPOT')   % creates D:\SPOT\MyProject instead
```

The name is also the diagram's name, so it must be a valid MATLAB name: letters, digits and _, starting with a letter, at most 63 characters, not a keyword and not an existing function. The project is a copy of `template` with the diagram renamed to `<name>.slx`, this guide, and an empty `Saved Data`; the template's settings, caches, build files, saved data and documentation are not copied, so the project starts from the default settings. It prints a link that opens the project and starts the console.

MATLAB in a Container

If MATLAB runs in a container (for example with `distrobox`), nothing extra is needed. SPOT fixes the shell MATLAB uses for `system` when the container doesn't set one, and opens the browser on the host.

2.5 First Start

1. In MATLAB, go to the project folder and type:

   ```
   SPOT          % start the console and open it in your browser
   ```

2. MATLAB starts the console's web server, connects to it, and opens the page **SPOT Local Simulations**. The address it prints includes a session key: the page only accepts requests from that link.
3. `SPOT open` opens the page again (for example after closing the tab). `SPOT stop` stops the console. Closing MATLAB also disconnects it.
4. Keep MATLAB open while you use the console: simulations, builds and the page's **MATLAB** command window all run in it. The **MATLAB** badge at the bottom of the page says whether it is connected, and what it is doing.

If...	then...
"needs Python 3.6 or newer"	Click **Install Python** in the dialog, run `spot.console.installPython`, install it (above), or point MATLAB at it: `pyenv('Version', 'C:\...\python.exe')`.
The page says it can't talk to MATLAB	Open it with the link MATLAB printed, or run `SPOT open`.
The browser doesn't open	Copy the address `SPOT open` prints into your browser.
Lab Disconnected (top right)	You're not on the lab network. Simulations and results still work; experiments and the live view need the lab network.
The console doesn't start	Its log is `.spot_cache/console.log` in the project folder.

3 The Local Console (SPOT Local Simulations)

3.1 Around the Page

- **Top bar.** The tabs **Simulation**, **Experiment** and **Results**. Next to them, the **Settings** group shows the file your settings are saved to (`spot_settings.json`) and has three buttons: **Load...** (load settings from a `.json`, or an old GUI state `.mat`), **Save As...** (save a copy) and **Reset...** (back to the template's defaults, see Section 3.5). On the right: **Lab Console** (opens the hardware console) and the lab badge (**Lab Connected** / **Lab Disconnected**).
- **3D view.** The table, the platforms and their paths. Its toolbar has **3D**/**Top**/**Side** views, **Initial Conditions** (**Off**, **Initial** or **Home**: which target poses to show on the table), **Labels** (name tags and +X/+Y/+Z labels) and **Axes** (each platform's body axes: +X red, +Y green, +Z blue). Drag to orbit, right-drag to pan, scroll to zoom. The **ZED2 Viewpoint** switch opens a window in the corner showing what the Chaser's ZED2 camera sees, with the ZED2's default HD720 intrinsics (1280×720, $f_x = f_y \approx 528$ px, 68.6° vertical field of view), from the centre of its **Left** or **Right** lens, or **Both** side by side (120 mm apart, 0.28 m up on its front face, the right lens 0.08 m right of centre, facing along its +X). It follows the Chaser live, in a replay or at its initial conditions, as the window's title says. The table is drawn as in the lab: the 0.33 m granite on its stand (a leg under each corner, painted the barrier's dark grey), with the barrier round its edges (from its CAD), lit from the ceiling, so a platform's shadow falls right under it. **Use CAD** draws the platforms from the lab's CAD instead of cubes: the Chaser in silver MLI with its ZED2 (Stereolabs' ZED 2i model, the ZED 2's housing, in a cutout of its size), the Garmin LIDAR-Lite behind the front panel's two small holes, and arm; every platform also shows its eight thrusters (Silvent MJ4 nozzles in black PLA inserts in the side panels' bottom notches), the LED mounts at its top corners and the countersunk black Phillips screws in its panels and aluminium frame (the links follow the joint angles in a replay), the Target in gold MLI, each with the docking cone and solar panels set in the Spacecraft Designer (Section 3.6), and the Obstacle as an aluminium frame with transparent blue plexiglass panels on its sides and top (every platform has its internal structure). The models are solid; a simulation laid over the live view is see-through with an outline, and loaded results are faint with a dashed outline. The models are made from the lab's SolidWorks assemblies by `build_cad_models.py`, which lives outside the template, with the CAD it reads (`designs/cad_tools`); run it again after changing the CAD.
- **Messages** (right, on the Simulation and Experiment tabs). Each platform's debug message, in three parts (RED, BLACK, BLUE): the text the diagram writes to `RED_Logger`, `BLACK_Logger` and `BLUE_Logger`, e.g. "Chaser: Running Phase 1". Only changes are listed, with their time and, when a message came several times in a row, how often (×**30**). After a simulation they are the simulation's messages, and the replay highlights the one in effect; during an experiment they come live from the platforms (in a diagram with the shared PhaseSpace clock), so a simulation shows what the experiment will. The arrow tucks the panel away to give the 3D view more room.
- **MATLAB command window** (bottom), with two tabs. **MATLAB**: type MATLAB commands. They run in your MATLAB session's workspace (Enter runs, Shift+Enter starts a new line, ↑ recalls earlier commands). Its tab shows what MATLAB is doing (**Idle**, or e.g. **running the simulation**); the arrow on the right collapses the window. **Live Plot** (Simulation and Experiment tabs): plots the signals the diagram's *Live Stream* blocks send, live in simulations and experiments. **Data Available to Stream** lists them, each named after what is connected to its block (e.g. `RED_Measured_States`); tick one to plot it, and choose its elements with the **1** **2** ... chips. The window (**10 s** to **All**), **Pause** and the readout under the mouse help to read it; after a simulation the replay's time is marked. To stream another signal, copy a Live Stream block in the diagram's *Live Data Streams* subsystem and connect a Data Store Read (or any signal) to it.

3.2 Simulation Tab

The sidebar has five tabs. Every change is sent to MATLAB at once and saved. MATLAB puts the settings into the base workspace as the diagram expects them, then runs `SPOT_User_Parameters.m`.

Scenario.

- **Diagram**: which `.slx` in the project folder to use. **Open in Simulink** opens it in MATLAB.
- **Platforms**: Chaser, Target, Obstacle and Arm (the arm on the Chaser).
- **Sample Rate**: the controller rate, in Hz. The diagram's step is 1/rate.
- **Data Rate**: the PhaseSpace data rate, in Hz: how often the cameras send the platforms' positions in an experiment (`dataRate` in the settings file; whole Hz, 1 to 960; 10 by default). It is the Experiment tab's **Data Rate** too, and *locked*: the padlock beside it unlocks it, after a warning, on both tabs (the page locks it again when reloaded). The diagram's `dataRate` variable is its period, 1/`dataRate` s; the LQR gains are designed at it and the measured states come at it, so build again after changing it.
- **Thruster PWM**: the thrusters' PWM frequency, in Hz (`PWMFreq` in the diagram; 5 by default). The duty cycles assume a valve period of 1/frequency. The Teensy firmware runs its own PWM at 5 Hz (`PWM_FREQUENCY` in `+spot/+hardware/teensy/teensy_pwm_control.ino`): change both, or the simulation warns. It is *locked* like the data rate: its padlock explains this before it unlocks.
- **Calibrated Thrust**: use each thruster's measured thrust (on by default; the lab's thruster calibration, Section 4.5) in the diagram's mixer, and so in the simulated plant and in experiments built from the project. Off, or without a calibration, every thruster gives the Platforms tab's **Thrust**, which says which is used.
- **Calibrated Arm**: use the arm's measured friction and inertia (on by default; the lab's arm calibration, or a `resources/arm_calibration.json` copied by hand: the newer). Off: the arm model's defaults.
- **Table Slope**: simulate the table's measured slope (on by default): the floating platforms slide downhill, and their air bearings drag them, as the lab's table slope calibration found (Section 4.5; the project's `resources/spot_calibration.json`). The line beside it says what the project has. Off, or without a calibration, the table is level.
- **Timeline**: the phases. Phase 0 synchronizes, 1 starts floating, 2 moves to the initial pose, 3 is the experiment (sub-phases 3.1–3.4), 4 returns home and 5 holds there. Type each duration in seconds. The switches (or a click on a segment of the bar) *skip* phase 2, any sub-phase or phase 4. A skipped segment takes no time, in simulation and in experiments. Phases 0, 1 and 5 always run (they switch the platforms on and off).

Platforms. Pick the Chaser, Target, Obstacle or Arm.

- **Initial Conditions**: where each platform starts (**Initial**: the simulation starts here and phase 2 drives the platform here) and where it returns (**Home**), as x [m], y [m], θ [°]. Click **Initial** or **Home** to show those poses on the table, then drag a platform to move it (Shift+drag rotates it).
- **Mass Properties**: the three scale readings A, B, C [kg] and the bifilar-pendulum test (T, L, D) give the mass, the centre of gravity and the inertia (pictures in `resources/`). **Override** uses a mass and inertia you type instead. **Thrust** is the force of one thruster.
- **Thruster Offsets and LED Positions**: the computed geometry. **Write PhaseSpace Marker File** writes `resources/tracker_positions_phasespace.txt`, which the Experiment tab sends to PhaseSpace before streaming.
- For the arm: its **Initial Pose** (shoulder, elbow, wrist angles), geometry, link masses and inertias (`resources/ArmConfig.png`).

Control. One LQR per platform. A row per axis (x, y, θ) holds its **Error Q**, **Rate Q** and **Effort R** weights. A larger Q corrects that error harder; a larger R uses the thrusters more gently. **Gain K (Computed)** shows

the resulting gain matrix. With the Arm on, two more cards set the arm's position and velocity controllers (Section 6): P, I, D per joint (the servos' own units), the profile and the PWM limit (`control.ARM` in the settings, `armControl` in MATLAB).

Solver. Fixed-step or variable-step, and its options. They are read from the diagram when you choose it, and used for simulations without changing the diagram file.

Workspace. MATLAB's base workspace: click a variable to inspect it (values, struct fields, or a plot for a time series).

Your First Simulation.

1. On **Scenario**, check the diagram, switch on **Chaser**, and leave the timeline as it is.
2. On **Platforms**, look at the Chaser's initial and home poses (show them with **Initial Conditions: Initial**).
3. Click **Run Simulation** (with **Real Time** on, it runs at the clock's pace, one second of simulation per second, to watch the Live Plot, the messages and the 3D view as during an experiment). The status line shows the progress, and **Run Simulation** becomes **Stop** while it runs. Simulink's messages go to the command window, not MATLAB's (warnings can be expanded there). If the simulation fails, the status line gives the root cause and the block it is in; **Details** opens the whole chain of causes, the line in your own code (e.g. `SPOT_User_Parameters.m`), what Simulink printed before failing and MATLAB's full report in the command window, and **Show Block** opens the diagram at the block, highlighted.
4. When it finishes, the replay starts in the 3D view. The playback bar has play/pause (also the space bar), a scrub bar with the phase boundaries, the current phase, the speed, and **Paths**. Paths are traced behind the platforms as they move.
5. **Save Results** (next to **Run Simulation**) keeps this run, with the settings it ran with, in `Saved Data/Simulation_<diagram>_<date_time>/`, to replay or plot later. Until then the run is only in MATLAB's workspace (`dataClass`, `simOut`).

Comparing Runs on the Table. **Compare** (in the playback bar) lays other results over the simulation: experiment runs, saved simulations, or copies of earlier runs.

- **Keep This Simulation** keeps a copy of the current run, so you can compare it with the next one.
- **Load Results...** opens the `Saved Data` list.
- Each result has a switch and a time offset [s] to line it up. Overlays are drawn with outlined platforms and dashed paths, labelled with the result's name.
- **Last Simulation** hides or shows the current run.

3.3 Experiment Tab

The Experiment tab builds the selected diagram on the lab console, deploys it to the platforms, runs it, and collects the data. It needs the lab network: without it the tab is locked and says so. It uses the Simulation tab's settings: the diagram, the platforms and the timeline.

Safety

Before starting an experiment, make sure the table is clear, the platforms are where they should be, and the E-stop is within reach. **Stop** sends "all off" to the platforms. Each platform's Teensy also switches everything off by itself if commands stop arriving for 2 s.

1. **Lab.** The card shows the lab console and PhaseSpace. **Lab Console (TX2) Address** is the TX2's address (`labConsoleAddress` in the settings file; 192.168.1.116): the page reaches the lab console there, experiments are built and deployed through it, and the platforms send it their clocks, debug messages and live streams (the diagram's `labConsoleIP`), so build again after changing it. The cameras' **Data Rate** [Hz] is the Scenario's **Data Rate** (the same setting, locked the same way), fixed while the cameras stream. **Start Cameras** sends this project's marker calibration (`resources/tracker_positions_phasespace.txt`) and starts streaming at that rate. The platforms then appear live in the 3D view.
2. **Platforms.** Each selected platform's state: **Ready** when it answers and its Teensy is free. Its bin button takes it out of the experiment (it is switched off in Scenario; the Chaser takes the arm with it).
3. **Build**, in three steps.
   (a) **Check**: loads the diagram and lists what the build will do; nothing is compiled.
   (b) **Connect**: connects MATLAB to the TX2. Type its password the first time; MATLAB remembers it.
   (c) **Build**: generates the code and compiles it on the TX2 (a few minutes). The build log appears live. The diagram file itself is not changed: the build's own settings (plant model commented out, ode4, NVIDIA Jetson board, build only) are undone afterwards.
4. **Deploy & run.**
   - **Deploy** copies the program from the TX2 to each selected platform.
   - **Start Experiment** starts it on all of them (PhaseSpace must be streaming). Each platform's clock appears over the 3D view, and the live paths are drawn. Tick **Simulation** there to overlay your simulation in step with the experiment's clock.
   - The template's diagram uses the *shared PhaseSpace clock* (see below): the platforms then wait at time 0 until every program is running, and the lab console starts them together 3 s later, through the PhaseSpace stream they all receive. The status says what it is waiting for, and then how close the platforms' clocks are ("clocks within 4 ms").
   - **Stop** stops it early. Otherwise it ends by itself at the end of phase 5.

   The Shared PhaseSpace Clock

   In the earlier template diagram (now in `archive/Legacy/original_diagram`) each platform broadcast its own Simulink clock and followed the last one to start, so the platforms were off by however their programs and the Wi-Fi happened to start. In the template's diagram, `Template_v5_0_0_2026a_Jetson_PSClock.slx`, the experiment time comes from the PhaseSpace stream instead (`spot.hardware.experimentTime`): one clock for every platform, started at an agreed moment, and a program started late stays at time 0. Its *Debugger* sends each platform's message to the lab console (the Messages panel), and records it in simulations, and its *Live Data Streams* subsystem feeds the Live Plot. Every UDP link in it uses `spot.hardware.udpSend` / `udpReceive` (the newest packet, never a queue of old ones).
5. **Data. Collect** copies each platform's log files into `Saved Data`, in a folder named after the diagram and the time with one subfolder per platform. It converts them to MATLAB time series (`<PLATFORM>_<n>.mat`, variable `dataClass_rt`) and saves the settings next to them. Open them on the Results tab.

Tip

If you change the timeline, the platforms or the diagram, build again: the settings are part of the program.

3.4 Results Tab

1. **Load data** (**Datasets** card).
   - **Last Simulation** takes a copy of the current run.
   - **Saved Data…** lists experiment runs ("Run 1", "Run 2" for each start) and saved simulations, newest first. Runs whose log files weren't converted yet have a **Convert ExpLog Files** button.
   - **Open .mat…** opens any file of logged signals.

   Each dataset's **3D** button replays it on the Simulation tab, over the simulation.
2. **Plot.** Click a dataset, then click its signals (grouped Chaser, Target, Obstacle, Arm, other) to add them to the selected plot; click again to remove. Multi-column signals (e.g. `Duty_Cycle`) have a button per column.
   - **+ Plot** adds a plot below. Each plot has one y axis and up to eight lines, and lines from different datasets can share a plot.
   - The **x** menu plots against another signal instead of time. A platform's `Py_m` against `Px_m` draws its path on the table; tick **Equal** for true shapes.
   - Hover to read values. Drag across a plot to zoom; double-click to zoom out. **Axes** sets the limits. **Link Time** zooms and hovers the time plots together. **Smoothing** applies a Gaussian filter.
3. **Export** (**Export** menu). MATLAB draws the plots in the lab's publication style: Times, the platforms' colours (Chaser grey-silver, Target gold, Obstacle blue, named so in the legend), every sample, and table plots with the platforms' outlines.
   - **Save PDF…** saves a PDF, EPS, SVG or 700 dpi PNG; choose its width, height and font size first.
   - **Open in MATLAB** opens it as a figure you can adjust.

3.5 Settings and Your Own Parameters

- The settings are saved to `spot_settings.json` after every change. Use **Save As…** to keep versions, and **Load…** to go back to one. **Load…** also reads a state saved by the old SPOT GUI.
- **Reset…** starts from a clean slate, in parts you choose.
  - *Simulation settings*: back to the template, keeping your diagram.
  - *Page and view*: the page's own preferences.
  - *Loaded results*: closes the loaded datasets.

  The settings file is backed up to `.spot_cache/settings_before_reset_<time>.json` first, so **Load…** can bring it back.
- Put variables of your own in `SPOT_User_Parameters.m`. It runs every time the settings are applied (and before every simulation and build), so it can also override any of SPOT's values.

3.6 Spacecraft Designer

The **Designer** tab, on both consoles, sets what is mounted on each platform. The 3D view shows the chosen platform's CAD model close up at the centre of the table; drag to orbit, scroll to zoom.

- Pick the platform (**Chaser**, **Target**, **Obstacle**) at the top of the sidebar.

- **Docking Cone**: **None**, **Male** (the cone that goes in, on the interface bracket, with the steel disk at its tip) or **Female** (the cup with the electromagnet at its bottom, on an arm coupler's plate). By default the Chaser has the male cone and the Target the female one, both at the left of their +Y face, as in the lab.
- On the Chaser, **Arm** mounts the robotic arm instead: on an arm coupler's plate (the female cup's bracket), with the male docking cone as its end effector, so it is the arm or a cone, never both. The place you pick is the arm's mount, and the view shows the whole arm at its initial pose. On the local console this is the same choice as the Simulation tab's **Arm** platform: picking **Arm** turns it on (and the simulation mounts the arm at that place), another choice turns it off, and switching the **Arm** platform on or off picks or drops it here (a dropped arm gives the Chaser back its male cone).
- Its place is one of 6: the +X (front, where the Chaser's ZED2 is), +Y or −Y face (not the back, −X, where the control panel is), and the left or right of the face as seen from outside, in the upper half. The places are symmetric about the face's vertical centre line (the arm coupler's place, mirrored). Click a place on the face map (each face drawn as seen from outside, with the faces its edges meet) or a disc on the model.
- On the Chaser, the orange place is the arm's usual one (the arm coupler); a cone there, on its own bracket, takes its place. The side panels follow the cone (or the arm): each face's panel has the cutout for the cone's bracket only where the cone is, so moving the cone leaves no empty cutout behind. Red, dashed places would be blocked by something that stands out of the face; you can still choose them, with a warning.
- **Solar Panels**: **Include** adds the pair of panels on their bracket over the top, standing off the ±X faces (**Along X**) or the ±Y faces (**Along Y**). The bend in one of the bracket's legs goes round the control panel (on −X). Each frame carries a 1 mm panel on each side, drawn as the lab's panels look (the CAD has the frames only): a white backsheet with rounded corners and an eyelet in each, round a field of 9×8 black monocrystalline cells with thin silver lines between them.

Each console keeps one design: the local console in `spot_design.json` in the project folder, the lab console on the TX2. Every change is saved at once, and every 3D view of that console draws it whenever **Use CAD** is on. On the local console, the **Lab Console** card says whether the lab console's design is the same, and offers **Send to Lab Console** and **Use Lab Console's Design**.

4 The Hardware Console (SPOT Hardware Interfaces)

On the lab network, open `http://192.168.1.116:8080` (or click **Lab Console** in the local console, which follows the Experiment tab's **Lab Console (TX2) Address**). It always runs on the TX2 and starts by itself when the TX2 boots. Everyone in the lab sees the same state: if someone else starts the cameras or switches a thruster on, your page shows it too.

4.1 PhaseSpace Tab

- **Compile** builds the PhaseSpace streamer (StreamData) on the TX2. It's only needed after StreamData's source changes, since `deploy_tx2.sh` already compiles it.
- **Stream**: choose a **Frequency** [Hz] and **TCP** or **UDP** (UDP drops a late frame instead of delaying the next ones), then **Start Streaming**. The platforms receive the positions on UDP port 31534. **Destinations** adds other computers that should receive them; **+ This Computer** adds yours.
- **Telemetry** lists each platform's position, attitude and rates, and the LEDs PhaseSpace can see. The 3D view shows the platforms and LEDs live (**LED IDs** numbers them; **LED Height Lines** draws a line from each LED down to the table).

The Experiment tab of the local console starts and stops the cameras for you, at the right rate, with your project's marker calibration.

4.2 Interactive Mode

Fly platforms by hand: drag where each should go on the table, and the console flies them there with the template diagram's own controller, several at once if you like.

1. Start streaming, and check on the Hardware tab that the platforms are **Ready**.
2. Switch on **Interactive Mode** (below **Telemetry**). It has a row for the **Chaser**, **Target** and **Obstacle**, each with a switch: switch on the ones to fly (or click them in the 3D view). Each one's *goal* appears on top of it, a dashed outline in its colour.
3. Drag each goal where its platform should go; Shift+drag turns it. **Here** puts it back on the platform.
4. **Fly to Goals**: every switched-on platform's pucks come on and they all fly to their goals at once. Keep dragging a goal and its platform follows. Switch on another platform during the flight and **Fly to Goals** adds it.
5. A platform's switch stops that platform (everything on it off); **Stop All** or **Esc** stops them all.

Each row shows the platform's goal, how far it still is from it, and each thruster's duty cycle; the 3D view draws the gas leaving the nozzles that fire.

Every `baseRate` (0.05 s at the default 20 Hz, as often as the diagram's controller and thruster code run), the TX2 takes each platform's pose and velocity from PhaseSpace, computes the forces with the default LQR law and the thrusters' duty cycles with the diagram's mixer, and sends them to its Teensy. Each platform has its own loop and its own controller process, so the platforms run side by side on the TX2's CPU cores. The law and the mixer are `spot.control.lqrCommand` and `spot.control.thrusterDutyCycles`, which hold verbatim copies of the blocks' code, ported line by line to Python (`+spot/+console/webapp/spot_console/control.py`) because the TX2 has no MATLAB. In MATLAB, `spot.control.verifyConsoleController` checks that the copies are still the diagram's code, that they reproduce a simulation of the diagram step by step, and that the Python agrees with MATLAB. The gains and mass properties are SPOT's defaults (**Controller** in the card lists them).

Safety

The platforms really fly. Clear the table first, and keep a hand near **Esc**.

- Each platform flies straight at its goal and knows nothing of the others. Goals must be at least 0.45 m apart (centre to centre), and away from platforms resting on the table, but the paths can still cross: place the goals so they don't.
- Everything on *one* platform goes off when PhaseSpace loses it (1 s), when it is seen off the table, when its link or Teensy is taken (an experiment, another program), or when someone switches it off on the Hardware tab. The others fly on.
- Everything on *every* platform goes off when the page that started the flight closes or loses the network (3 s), or on **Stop All** / **Esc**.
- Goals stay 0.2 m inside the table's edges. Any page can stop a flight; only the page that started it can move its goals or add platforms. The Hardware tab's buttons for a flying platform are locked meanwhile.

4.3 Hardware Tab

Each platform has a card.

- **Pucks** switches its air bearings on or off.
- The eight thruster buttons fire one thruster each, at the duty cycle set beside them.
- **Thruster Check** fires them one after the other, for the time set per thruster.
- **Esc** switches everything off on every platform.

Commands go to each platform's Teensy over SSH. The Teensy switches everything off when commands stop for 2 s, so nothing stays on if the page or the TX2 goes away. While an experiment uses a platform's Teensy, its card is locked.

4.4 For Lab Administrators

These one-time scripts live in `+spot/+console/deploy/` and `+spot/+hardware/teensy/`. Run them from a computer whose SSH key the TX2 and the platforms accept, and start the path with `./` (bash reads a path starting with + as an option).

`./+spot/+console/deploy/deploy_tx2.sh`	Copy the console to the TX2, compile StreamData there, and (re)start its service. Run it after any change to the console.
`./+spot/+console/deploy/setup_hardware_link.sh`	Install the platform agent on RED/BLACK/BLUE and give the TX2's key access to it (only to it).
`./+spot/+hardware/teensy/setup_teensy_serial.sh`	Make each platform's Teensy serial port usable without sudo.

4.5 Calibration Tab

Measurements of the lab that make simulations behave as the experiments do. The lab console runs them and keeps the lab's calibration (`~/.spot_calibration.json` on the TX2); each project keeps a copy, `resources/spot_calibration.json`, which its simulations use. Three calibrations: **Table Slope**, **Thrusters** and **Arm**.

Table Slope. The table isn't perfectly level, so a floating platform slides downhill. With PhaseSpace streaming and the platform resting on the table, the calibration

1. records the platform resting for 2 s (the cameras' noise);
2. floats it with the thrusters off, and records it drifting from where it is;
3. flies it (the diagram's own controller, as Interactive Mode) to each point of a grid over the table, lets it settle, switches the thrusters off and records it drifting again;
4. fits the table's height surface (a smooth quadratic surface, or a plane when the drifts don't cover enough of the table) and the air bearings' drag to every drift.

Choose the platform, the grid (3×2 by default), the drift time (12 s), the edge margin (0.35 m) and the model, then **Start Table Calibration**; the default takes about 4 minutes. The map then shows the table's height in colour (blue low, red high), its contour lines, which way is downhill and the drifts; below it, the tilt at the centre and the steepest (in mrad: 1 mrad is a 1 mm drop per metre), the downhill direction, and how well the model fits. The 3D view's **Table Slope** switch colours the table top the same way.

Safety

- A drift ends at once (pucks off: the platform stops on its pucks) when the platform comes within the edge margin of an edge, within 0.45 m of another platform, or goes faster than 0.15 m/s. Grid points next to other platforms are skipped; clear the table around the grid.
- **Stop** or **Esc** on the tab ends it with everything off; so does Esc or switching the platform off on the Hardware tab (where it shows as *Calibrating*), PhaseSpace losing it for 1 s, or the link to it dropping. Experiments and Interactive Mode can't start while it runs, nor it while they do.

Thrusters. Each thruster's real thrust. With PhaseSpace streaming and the platform resting on the table, for each thruster 1 to 8 (and each repeat) the platform floats where it rests, the thruster fires at full thrust for the burn time (2 s by default), the diagram's controller flies it back to where it started, and the pucks go off for a moment's rest. Each burn's thrust is the platform's mass times the acceleration it gave (the table's slope, from the table calibration, taken out), so it is only as good as the platform's mass, inertia and centre of gravity: before it starts, a dialog shows the values the lab console assumes (SPOT's defaults), lets you change them for this run, and asks you to confirm. The result shows each thruster's thrust against nominal (bars with their spread, and arrows on the platform showing which way each really pushes), and a *torque ratio*: the turn each burn gave against the turn its thrust should give, which is far from 1 when the CG or the inertia is off. The lab console's controller (Interactive Mode) uses the calibrated thrust at once.

Arm. The arm's friction and inertia, with the real servos: the tab runs `calibrate_arm.py` (Section 6) on the Chaser through its agent. Opening **Arm** checks what the script needs there (Python 3, the Dynamixel SDK, the U2D2 at `/dev/ttyUSB0` and access to it, nothing else holding it), item by item with what to do about each. Pick the joints and tests, then **Start Arm Calibration**; a dialog asks you to confirm the Chaser rests on the table and the arm is clear (each joint moves up to 25°). The console keeps the Chaser resting meanwhile; **Stop** (or Esc) switches the servos' torque off at once. The result shows each joint's friction (its breakaway torque, and the friction against speed with the fitted curve), its inertia, and its position steps. **Rehearse (Simulated Servos)** runs the script's simulated servos instead: no hardware needed, and the result is never saved as the lab's.

On the local console. The Calibration tab shows the project's calibration next to the lab's, with **Update from Lab**. Whenever your computer is on the lab network and the lab's calibration is newer, the page asks once: **Update** copies it into the project, **Not Now** keeps the project's (and doesn't ask again until the lab's changes), and **Never Ask Again** stops asking for this project (the tab's **Ask When the Lab's Is Newer** turns it back on). The Simulation tab's **Table Slope**, **Calibrated Thrust** and **Calibrated Arm** use it.

5 Running Simulations Without the GUI

Everything the console does is an ordinary MATLAB function (Appendix A), so you can script it: for batch runs, parameter sweeps, or when you just prefer the Command Window. Run these from the project folder.

5.1 One Simulation

```
root = pwd;                                        % the project folder
s = spot.params.load('spot_settings.json');        % the console's settings (or spot.
    params.defaults())
s.diagram = 'Template_v5_0_0_2026a_Jetson_PSClock.slx';
s.platforms.RED = true;  s.platforms.BLACK = false;
s.timing.subPhases = [10 10 30 60];                % phase 3 sub-phases [s]
s.timing.on.phase4 = false;                        % skip phase 4
```

```
s.initial.RED.initial = [2.0 1.2 90];           % x [m], y [m], theta [deg]
s.control.RED.lqr.Q = [2 2 0.1 10 10 1];        % x y theta dx/dt dy/dt dtheta/dt
s.solver = spot.sim.readSolver(s.diagram, root, s.solver);   % the diagram's own solver
out = spot.sim.run(s, root);                    % simulates; dataClass is in the
    workspace
fprintf('%g s simulated in %.1f s\n', out.tsim, out.seconds);
```

`spot.sim.run` applies the settings, runs `SPOT_User_Parameters.m`, simulates without modifying the diagram file, and leaves `dataClass`, `dataPacket` and `simOut` in the base workspace, as the old GUI did. The settings are an ordinary struct: see `spot.params.defaults` for every field and its unit.

5.2 Looking at the Results

```
plot(dataClass.RED_Px_m); hold on; plot(dataClass.RED_Px_Desired_m)   % timeseries
spot.data.saveSimulation(s, root);              % keep it in Saved Data (with s)

% A figure in the lab's publication style, saved as PDF
ds = spot.data.load(struct('source', 'simulation'), root, s);   % a copy of this run
lookup = @(id) ds{1};
panel = struct('traces', struct('id', 'sim', 'signal', {'RED_Px_m', 'RED_Px_Desired_m'
    }, 'col', 1));
fig = spot.data.toFigure({panel}, lookup);
spot.data.exportFigure(fig, fullfile(root, 'red_x.pdf'));
```

A panel whose `x` is `struct('signal', 'RED_Px_m', 'col', 1)`, with `RED_Py_m` as its trace, draws the Chaser's path on the table, with its outline at the start and the end. Experiment runs load the same way: `spot.data.listSaved(root)` lists them, and `spot.data.load` reads one (its help explains the `files` and `file` forms).

5.3 From a Terminal (Batch)

MATLAB's `-batch` option runs a command without the desktop, and exits when it ends. The same line works on Windows (Command Prompt or PowerShell) and Linux:

```
matlab -batch "cd('C:/Users/you/SPOT/MyProject'); s = spot.params.load('spot_settings.
    json'); spot.sim.run(s, pwd); spot.data.saveSimulation(s, pwd);"
```

For anything longer, put the commands in a script (e.g. `my_runs.m` in the project folder) and run `matlab -batch "my_runs"`.

5.4 A Parameter Sweep

```
root = pwd;
s = spot.params.load('spot_settings.json');
s.solver = spot.sim.readSolver(s.diagram, root, s.solver);
for q = [0.5 1 2 4]
    s.control.RED.lqr.Q(1:2) = q;               % position weights
    out = spot.sim.run(s, root);
    folder = spot.data.saveSimulation(s, root); % each run in its own Saved Data folder
    fprintf('Q = %g: saved in %s\n', q, folder);
end
```

The saved runs then appear in the Results tab's **Saved Data...** list, ready to plot or replay side by side.

5.5 Experiments from MATLAB

Building works from MATLAB too, on the lab network:

```
p = spot.experiment.plan(s, root);              % what the build would do (no compiling
    )
spot.experiment.connect(root, 'tx2-password');  % once; afterwards spot.experiment.
    connect(root)
out = spot.experiment.build(s, root);           % generates the code and compiles it on
     the TX2
```

Deploying, starting, stopping and collecting the data are done by the lab console: use the local console's Experiment tab (or the hardware console) for those steps.

6 The Arm: Servos, Friction and Calibration

The Chaser's arm has three Dynamixel MX servos (shoulder, elbow, wrist; IDs 1, 2, 3). Their gearboxes have a lot of friction: a joint takes a firm push to start moving, a small torque command doesn't move it at all, and the arm stays still while the platform flies.

The servos run in **current control** for the whole experiment: the diagram sends each joint a torque, and the servo holds the motor current that gives it. Position and velocity control are controllers in the diagram that run the servos' own control laws (the ones the servos run in their position and velocity modes) and give torques. In simulation the plant stands in for the arm, with the same inputs and outputs.

6.1 How the Arm Is Simulated

The diagram's **Simulate Plant Dynamics / RED and ARM Dynamics / RED + ARM** holds one block, **RED + ARM Plant** (`spot.arm.Plant`). Every step of the diagram (1/**Sample Rate**, e.g. 50 ms), it takes 1 ms steps of its own of:

- **The servos** in current control, with the torques the diagram sends (`ARM_Controls`, N m): each becomes a goal current as the hardware block converts it (torque / torque constant, in 3.36 mA steps, within the Current Limit), and the servo holds that current as far as its supply voltage allows (the motor's back-EMF), from the datasheet's stall torque, stall current and no-load speed.
- **The dynamics** of the base and the three joints, coupled, with each joint's rotor inertia.
- **The friction** of each joint (below).

Every step it also writes what the hardware block measures: the joint angles at the encoders' resolution and the joint speeds in the servos' velocity units (`ARM_Measured_States`), and the torques the servos report (`ARM_Measured_Torque`). The rest of the diagram keeps its own rate: a 275 s simulation of all three platforms and the arm takes about 10 s.

6.2 The Arm's Controllers

3 - Control picks the arm's controller from `ARM_Control_Law_Enabler`, which each phase of **1 - Experiment Logic** sets (Table 5). Each one writes the joint torques to `ARM_Controls` and its number to `ARM_Control_Mode`.

Gains at 20 Hz. Run every millisecond with the servo's gains (Position P/I/D 400/0/200), these controllers are the servos' own control laws, and `spot.arm.compareSteps` checks that against the real servos. In the diagram they run once a step (50 ms at 20 Hz), where the servo's gains make the arm unstable, so their

Enabler	Controller	What It Does
1	Default Position Control (ARM)	The servo's position law (`spot.arm.PositionControl`): the path's joint angles [rad] are goal positions; the desired position moves to them along the profile (Profile Velocity 20 = 0.48 rad/s), and a PID on the error gives the PWM the servo would set, as the torque the motor would give.
2	Custom Control (ARM)	Torque control, an example: the path's three values go straight to the servos as joint torques [N m]. Put your own control law here.
3	Default Velocity Control (ARM)	The servo's velocity law (`spot.arm.VelocityControl`): the path's three values are goal joint speeds [rad/s]; a PI on the speed error gives the torque.
other	Disable Control (ARM)	No torque: the joints only have their friction.

Table 5: The arm's controllers. Their gains, profiles and PWM limits are set in the **Control** pane (with the Arm on).

defaults are retuned for 20 Hz: for the three joints together (they are coupled), over the arm's poses, with the base fixed or floating, the rotor inertia ±50 % and the friction from 0.3 to 2 times the defaults. Position: P 25/56/21, I 10, D 1545/230/236 (shoulder/elbow/wrist); velocity: P 160/70/30, I 220/1500/300. That is about 1 N m/rad of stiffness, a twentieth of the servo's own, so friction holds a joint a few degrees short until the I gain works it off (within about 3° after a few seconds with the default friction). A faster loop would allow the servo's stiffness back: the controllers take their step (**StepTime**) as a parameter, and the bus can serve about 250 reads a second (Section 6.5).

Joint angles are the servos' own: each joint is 0 straight out (the shoulder: out from the face its mount is on), and the servos take ±90° to either side (Min/Max Position 1024–3072). The position controller ignores a goal beyond that, as the servo's position mode does, so the joint holds its last goal. In current control the servos have no limits of their own, so the hardware block (and the plant) sends no torque that pushes a joint further beyond them. After a simulation, the command window lists such goals and torques as warnings (`spot.arm.checkCommands`), and torques beyond what the Current Limit allows. **The mount** is the Designer's (Section 3.6), in the settings as `s.mass.arm.mount`; **Arm Geometry**'s Φ and b_0 are for the usual place (+Y face, left), and `spot.arm.mountGeometry` moves them with the mount.

6.3 The Friction Model

For each joint: at rest, the joint stays at rest until the torque on it exceeds its breakaway torque T_s (`static`). Moving at speed v, the friction against the motion is

$$T_c + (T_s - T_c)\, e^{-(v/v_s)^2} + b\,|v|$$

with the Coulomb torque T_c (`coulomb`), the Stribeck velocity v_s (`stribeck`) and the viscous coefficient b (`viscous`). Each 1 ms step solves for the friction torques that keep the joints at rest if they can, and otherwise applies the sliding friction, at the end of the step, so the joints stick and slip without chattering. The values are in `armModel` (`help spot.arm.defaults`): placeholders (T_s 0.30 N m, T_c 0.20 N m) until the arm is calibrated. Change them in `SPOT_User_Parameters.m`, e.g. `armModel.friction.static = [0.4 0.3 0.25];`.

6.4 Calibrating the Friction in the Lab

`resources/arm_calibration/calibrate_arm.py` measures each joint with the real servos (Python, with the Dynamixel SDK only: `pip install dynamixel-sdk`). The usual way is the lab console's **Calibration** tab (**Arm**; Section 4.5), which runs it on RED for you and keeps its result as the lab's. It can

also be run by hand on RED's computer (the arm's adapter is `/dev/ttyUSB0`) while no SPOT program runs:

```
python3 calibrate_arm.py                    # all joints, all tests (about 15 minutes
    )
python3 calibrate_arm.py --joints 3 --tests breakaway sweep
python3 calibrate_arm.py --simulate         # a rehearsal, without hardware
```

Safety

The platform must rest on the table (not float), and each joint moves up to 25° (`--window`) from where it is: start with the arm clear and away from its limits. Ctrl+C stops at once, with every servo's torque off.

For each joint, the other two holding their position, it runs:

1. **Breakaway**: current ramps from zero, three each way, until the joint moves: T_s.
2. **Speed sweep**: constant speeds back and forth (velocity mode), the friction torque at each from the motor current: T_c, v_s and b. Speeds where the joint stick-slips are left out.
3. **Inertia**: a torque step from rest; the acceleration gives the joint's inertia, and SPOT takes off the links' part to get the rotor's (`armature`).
4. **Position steps** with SPOT's servo settings, to check the simulated servo (below).

It writes `runs/<date_time>/arm_calibration.json` and the raw data (CSV). From the Calibration tab, projects get it with **Update from Lab**; run by hand, copy the JSON file to the project's `resources` folder. Either way, from the next simulation `armModel` uses the newer of the two (see `spot.arm.parameters` and `spot.arm.loadCalibration`; the Scenario's **Calibrated Arm** switches it off). `--fit runs/<date_time>` fits saved data again. Torque is the motor current times the torque constant from the servo's datasheet (MX-64: 1.46 N m/A; `--kt` to change it).

Checking the servo model: `spot.arm.compareSteps(folder, armModel)` replays the run's position steps through the servo's position law (`spot.arm.PositionControl` every millisecond, with the servo's gains) driving the plant, and plots the simulated motion over the measured one, with the difference. A large difference points at its settings (`armModel.servo`), the supply voltage (`armModel.motor.voltage`) or the friction.

6.5 The Arm on the Hardware

On RED, the **Dynamixel Arm (Torque)** block (`spot.hardware.ArmTorque`, in **Manipulator Encoder Data**) drives the servos. Its first step sets them up (torque off, current control, Current Limit, goal current 0, torque on). Then, every step, it reads each servo's position, speed and current before the controllers run (priority 21), and sends the step's torques (`ARM_Controls`, handed over by **Robotic Arm Code**) at the end of the step, so a torque goes out in the step it was computed, as in the simulation. At the end of the run it turns the torque off. Besides the angles and speeds it logs the measured torques (`ARM_*_Torque_Nm`) and the bus's status: the servos read that step (`ARM_Servos_Read`, 3 when all answered), the read's time [ms], the USB adapter's latency timer [ms], the reads missed since the start, and a servo's hardware alert (e.g. overload; the servo then turns its torque off). The old **Control Dynamixel Actuators** and **Read Joint Positions** blocks are kept in `+spot/+hardware`, out of the diagram.

The slow encoder readings. With the old blocks the joint angles came in at a few hertz, although the servos update them every millisecond. The arm's USB adapter (U2D2, an FTDI chip) passes on what it receives only when its latency timer runs out, 16 ms by default on Linux, and the old code gave up on the servos' replies after 18.6 ms: a read succeeded only when nothing else delayed it, one servo's late reply discarded

all three, and the old block then repeated the last angles and reported zero speed without saying so. Against a stand-in bus with that timing, the old code read at 20, 16, 12 and 6 Hz (of 20) with up to 0, 6, 10 and 30 ms of extra USB or scheduling delay. The new block asks the adapter for a 1 ms timer, waits for the replies without spinning, keeps each servo's reply on its own, and says in its status what it missed: on the same stand-in every read arrived, in about 3 ms. Run `+spot/+hardware/arm/setup_arm_serial.sh` once; it makes the 1 ms timer permanent on RED with a udev rule. The block's **ZeroReturnDelay** option sets each servo's Return Delay Time to 0 (an EEPROM change; 500 μs each by default) for faster reads.

7 Inside the Diagram: The Order of a Step

Every step (every 0.05 s at 20 Hz), Simulink runs each block of the diagram once, in an order it works out before the simulation starts. Most of the time you never think about that order. In SPOT's diagram you have to, and this section explains why, what the diagram does about it, and what to do when you change the diagram. The template's diagram (`..._PSClock.slx`) is set up this way, and a new project's diagram, copied from it, starts out the same.

7.1 Why the Order Matters in SPOT

Simulink orders blocks by their *signal lines*: a block runs after the blocks that feed it. SPOT's top-level subsystems are not connected by signal lines at all. They pass everything through *data stores*: a block writes a value with a *Data Store Write* (e.g. 1 - Experiment Logic writes `RED_Desired_States`), and other blocks read it with a *Data Store Read* (2 - Guidance and 3 - Control read it). A data store is like a global variable: it carries no "run me first" information, so Simulink is free to put the reader before the writer. Then, in each step, the reader gets the value from the *previous* step.

That is what happened. Simulink ran 2 - Guidance and 3 - Control before 1 - Experiment Logic (so they acted on the phase one step late), the thruster code before the experiment time and the measurements were written, and the data logger before most of what it logs. Simulink noticed, and said so in its warnings:

- "The block ... is reading from the data store ... *before any blocks have written* to this entire region of memory": a reader ran before the writer.
- "The block ... is writing to the data store ... but the block(s) ... *have already read*": the same thing, seen from the writer.

7.2 Virtual and Atomic Subsystems

A subsystem (the grey box you draw around blocks) comes in two kinds:

Virtual (the default). Only a drawing aid. When Simulink works out the order it removes the box and places each block inside it one by one, among all the other blocks of the diagram. The blocks of "1 - Experiment Logic" can end up anywhere in the step, mixed with those of "3 - Control". A virtual subsystem cannot be given a place in the order.

Atomic (**Treat as atomic unit** ticked). Runs as *one unit*: all its blocks, together, at one place in the step. Inside it Simulink still orders the blocks by their lines, but from the outside it is a single block, and like any single block it can have a *priority*.

A *priority* is a number on a block (or atomic subsystem): where the order is not already decided by signal lines, a lower number runs earlier. Blocks without one are placed wherever Simulink likes. SPOT's top-level subsystems are now atomic, with priorities in the order of a real step.

Where to Find These in Simulink

- **Atomic:** right-click the subsystem → **Block Parameters (Subsystem)** → tick **Treat as atomic unit**.
- **Priority:** right-click the block → **Properties** → **General** → **Priority** (a number; lower runs first).
- **See the order:** **Debug** tab → **Information Overlays** → **Execution Order**. After an update (Ctrl+D) each block shows its place as `s:n`: system `s`, `n`-th. At the top level of SPOT's diagram you see 1, 2, 3, . . . on the subsystems, in the order below.
- **From MATLAB:** `set_param(block, 'TreatAsAtomicUnit', 'on', 'Priority', '30');` the order of a compiled model: `get_param(block, 'SortedOrder')`.

7.3 SPOT's Order

Figure 2 shows one step as the diagram now runs it, and Table 6 the priorities (the gaps leave room for subsystems you add).

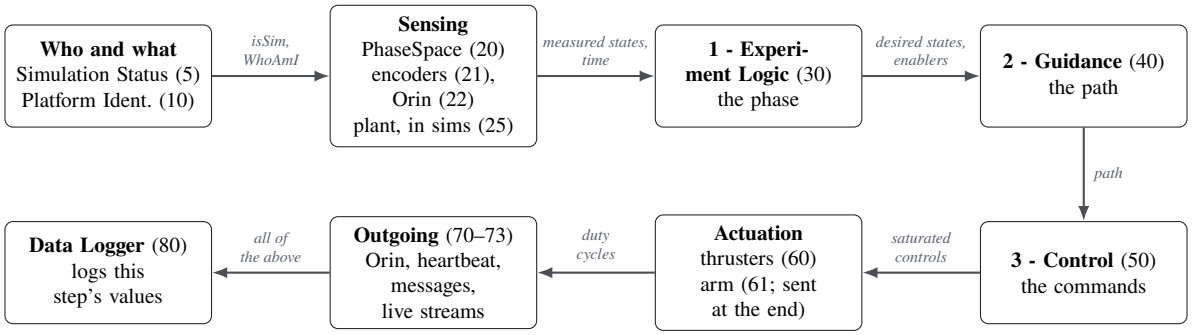


Figure 2: One step of the diagram, in order (priorities in brackets). Each arrow is a data store written on its left and read on its right, so every reader runs after its writer.

Table 6: The top-level subsystems' priorities in the template's diagram.

Priority	Subsystem	Writes (what later subsystems read)
5	Simulation Status	`isSim`
10	Platform Identification	`WhoAmI`
20	PhaseSpace Camera Code	`Univ_Time`, `*_Measured_States` (experiments)
21	Manipulator Encoder Data	`ARM_Measured_States`, `ARM_Measured_Torque` (experiments; the arm's torques go out at the end of the step)
22	Receive Data from Orin	`VIS_Measured_States`, `VIS_Confidence`
25	Simulate Plant Dynamics	`*_Measured_States` (simulations); see Section 7.5
30	1 - Experiment Logic	`*_Desired_States`, enablers, planner selections, `Float_State`
40	2 - Guidance	the path to follow
50	3 - Control	`*_Saturated_Controls`, `ARM_Controls`
60	Thruster & Float Control	duty cycles, Teensy commands
61	Robotic Arm Code	hands the step's `ARM_Controls` to the arm's block
70–73	Send Data to Orin, Check Connection, Debugger, Live Data Streams	(outgoing UDP)
80	Data Logger Subsystem	the ExpLog files (experiments), `dataPacket` (simulations)

The same order is used in the program built for the platforms: the generated code runs the subsystems in this order every step. *Simulate Plant Dynamics* is an *If action* subsystem (it runs only when its If block says

so, i.e. in simulations), so it runs where its If block runs: its priority is set on that If block (and on the Data Store Read of `simMode` that feeds it).

7.4 Demonstration

`documentation/demo_execution_order.m` builds a tiny diagram in memory (nothing is saved) that works like SPOT's: "1 - Experiment Logic" writes the phase into a data store and "3 - Control" reads it, with nothing else connecting them. The phase changes from 0 to 1 at $t = 0.5$ s; the model runs every 0.1 s. It simulates three versions:

```
addpath('documentation')
demo_execution_order                      % prints the three cases
demo_execution_order('order_demo.pdf')    % and saves the figure below
```

It prints (this is a real run):

```
A  virtual subsystems (Simulink picks the order)
   ran first: 1 - Experiment Logic
   warnings:  0
   3 - Control saw phase 1 at t = 0.50 s (in the same step)

B  atomic, priorities wrong way round (Control 30, Logic 50)
   ran first: 3 - Control
   warnings:  1
              - The block '3 - Control/Read phase' is reading from the data store
                'Phase store' before any blocks have written to this entire region
                of memory at time 0.0.
   3 - Control saw phase 1 at t = 0.60 s (1 step late)

C  atomic, priorities in step order (Logic 30, Control 50)
   ran first: 1 - Experiment Logic
   warnings:  0
   3 - Control saw phase 1 at t = 0.50 s (in the same step)
```

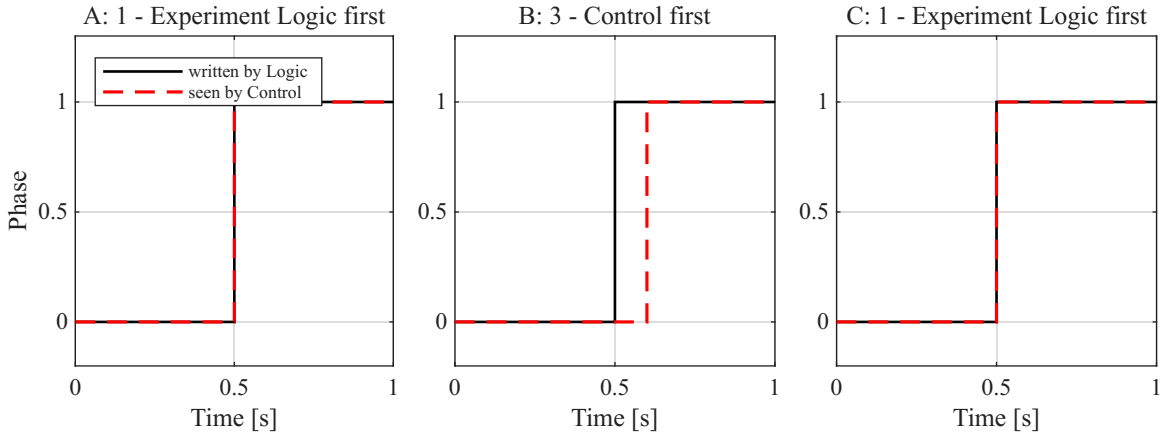


Figure 3: What 3 - Control saw (dashed) against the phase 1 - Experiment Logic wrote (solid). In B, Control ran first, so in every step it read the value from the step before: the change reached it 0.1 s late.

A is how SPOT's diagram was: virtual subsystems, Simulink's choice. In this tiny model it happened to choose the right order. Nothing guarantees that: in SPOT's diagram, with hundreds of blocks, it chose Control before Experiment Logic. That is the problem with leaving it to Simulink: it may be right today and wrong after an unrelated edit.

B forces the wrong order with priorities, to show what it does: Simulink's warning, and Control one step behind (Figure 3). This is what SPOT's 2 - Guidance and 3 - Control did.

C is what SPOT's diagram does now: atomic subsystems, priorities in the order of a step. No warning, and Control sees the phase in the step it changes.

In SPOT's own diagram, putting the subsystems in order changed the simulated motion by at most a few centimetres in the middle of manoeuvres (the phase logic no longer lagging a step), and removed all of Simulink's data store warnings.

7.5 The Plant Loop, and Stores That Hold a Value

In a simulation the data stores form a loop: the plant writes the measured states, Control reads them and writes the commands, and the plant reads the commands (Figure 4). In a loop, *one* store has to be read before it is written in each step, whatever the order. SPOT puts the plant with the sensing, as PhaseSpace is in an experiment, so Control gets this step's measurements and the plant uses the previous step's command for one step (as if the command were held). Putting the plant last instead would give Control the previous step's measurements; the loop has the same one-step delay either way.

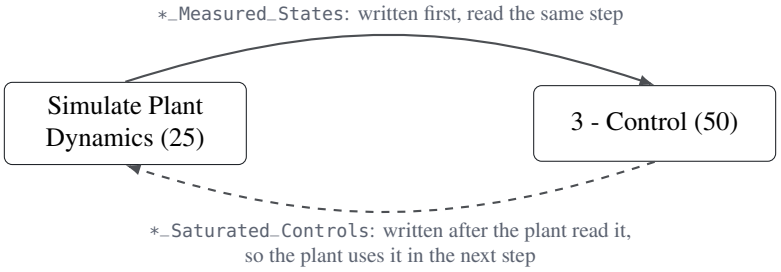


Figure 4: The simulated plant's loop. The dashed store is the one read before it is written.

Simulink would warn about that store every time, so its check is turned off, on that store only. A few other stores are also meant to be read while they hold a value. Simulink's check is a setting of each *Data Store Memory* block (double-click it → **Diagnostics**: **Detect read before write**, **Detect write after read**); for the stores in Table 7 it is off, with the reason in the block's **Description** (right-click → **Properties**). For every other store it stays on, so a new ordering mistake is still reported.

Table 7: The data stores whose "read before write" check is off, and why.

Data stores	Why reading them before (or without) a write is intended
`*_Saturated_Controls`, `ARM_Controls`	The plant loop above (also "write after read").
`*_Desired_States`	Written only in the phases that use them; they keep their value in between.
`WhoAmI`, `VIS_*`, `ARM_Measured_States`	Written only in experiments (or when the arm is simulated); otherwise their initial value.
`*_IMU_Measurements`	Nothing writes them yet (placeholders the logger records).

7.6 When You Change the Diagram

- **A new top-level subsystem** that reads or writes data stores: tick **Treat as atomic unit** and give it a priority from Table 6 that puts it after what it reads and before what reads it (e.g. a new sensor 23, an extra guidance step 45). Without one, Simulink places it anywhere.
- **A new data store warning** after a simulation (the console's command window shows "Simulation finished . . . *N* warnings"; click it): find the reader and the writer it names. If the reader should run

after the writer, fix the order with priorities. If the reader is meant to see the previous or initial value (like Table 7), turn that store's check off and say why in its description.

- **Inside a subsystem**, the same rules apply, one level down: an atomic subsystem orders its own blocks by their lines, and you can give its inner atomic subsystems priorities too.

A Function Reference

Every MATLAB function SPOT provides, by package. In MATLAB, `help spot` lists the packages, `help spot.<package>` a package's functions, and `help spot.<package>.<function>` the details. Optional inputs are shown in square brackets.

spot.arm – The Simulated Arm: Servos, Friction, Calibration

Call	Purpose
`msgs = spot.arm.checkCommands(dataClass, armModel)`	Arm goals the controllers ignore and torques the servos can't give, from a run's log.
`r = spot.arm.compareSteps(folder, armModel[, show])`	The simulated servos against the real ones, on the calibration's position steps.
`C = spot.arm.coriolisMatrix(p, x)`	Coriolis and centripetal matrix of the Chaser (RED) with its three-link arm.
`a = spot.arm.defaults()`	The simulated arm's servos, their friction and the plant's internal step.
`M = spot.arm.inertiaMatrix(p, x)`	Mass matrix of the Chaser (RED) with its three-link arm, free floating.
`a = spot.arm.loadCalibration(file, a)`	Apply the arm calibration (resources/arm_calibration/calibrate_arm.py) to a model.
`arm = spot.arm.mountGeometry(arm)`	The arm's geometry for where it is mounted on the Chaser.
`a = spot.arm.parameters(v[, root])`	The arm model spot.arm.Plant simulates (the workspace variable armModel).
`p = spot.arm.Plant('Model', armModel[, 'FixedBase', true]); [pos, vel, torque] = p(F_RED, jointTorques)`	The Chaser with its arm and three Dynamixel servos in current control (with friction), integrated at 1 ms within each diagram step (the RED + ARM Plant block).
`c = spot.arm.PositionControl('Motor', armModel.motor, 'StepTime', baseRate); tau = c(goal, [q; dq])`	The servos' own position control law (profile, PID), run in the diagram and giving joint torques (the Default Position Control (ARM) block).
`servo = spot.arm.servoFromDiagram(model, servo)`	The simulated servos' limits, from the diagram's arm blocks.
`c = spot.arm.VelocityControl('Motor', armModel.motor, 'StepTime', baseRate); tau = c(goalSpeeds, [q; dq])`	The servos' own velocity control law (PI), run in the diagram and giving joint torques (the Default Velocity Control (ARM) block).
`tau = spot.arm.withinLimits(tau, pulses, minPosition, maxPosition)`	No joint torque that pushes a joint further beyond its position limits.

spot.console – Starting and Stopping the Local Console

Call	Purpose
`spot.console.ensureShell()`	Make sure MATLAB's system() has a working shell.
`exe = spot.console.findPython()`	A Python 3.6+ to run the SPOT Console's web server with.

Call	Purpose
`[ok, message] = spot.console.installPython([dryRun])`	Try to install Python 3 for the SPOT Console (Windows or Linux).
`exe = spot.console.offerPython()`	Ask to install Python when the SPOT Console can't find it; " if it still can't.
`url = spot.console.open(root[, show])`	Open (or just show the address of) the project's running SPOT Console.
`ok = spot.console.openBrowser(url)`	Open a URL in the computer's default web browser.
`removed = spot.console.removeOldLibraryPaths()`	Take old Custom_Library folders off the MATLAB path.
`url = spot.console.start(root)`	Start the SPOT Console for a project and connect this MATLAB session to it.
`ready = spot.console.status(root)`	The project's running SPOT Console (ports and token), or [] if none answers.
`spot.console.stop(root)`	Disconnect the MATLAB worker and stop the project's SPOT Console web server.

spot.control – Controller Design

Call	Purpose
`u = spot.control.lqrCommand(K, measured, desired, desiredRate)`	The diagram's default LQR control law: forces and torque for one platform, one step.
`K = spot.control.lqrGains(sys, Q, R)`	Discrete LQR gain matrix for a platform.
`F = spot.control.nominalForces(force)`	Thrust of each of the 8 thrusters of a platform.
`sys = spot.control.stateSpace(mass, inertia, Ts)`	Discrete planar rigid-body model of one platform.
`[duty, Fb, H] = spot.control.thrusterDutyCycles(u, theta, thrusterDist, F[, maxIters, tol])`	The diagram's thruster mixer: the 8 thrusters' duty cycles for a force command.
`report = spot.control.verifyConsoleController(varargin)`	Check the lab console's Interactive-mode controller against the diagram.

spot.data – Results: Saved Runs, Signals, 3D Playback, Figures

Call	Purpose
`b = spot.data.bodies(ds)`	Which platforms (and the arm) a dataset can show on the table.
`file = spot.data.exportFigure(fig, file[, opts])`	Save a figure for a paper or thesis, as the lab's GetStarted template does.
`signals = spot.data.flatten(x[, prefix])`	Logged data as one flat struct of timeseries.
`runs = spot.data.listSaved(root)`	What is in the project's "Saved Data" folder, for the console's Results page.
`datasets = spot.data.load(spec, root[, settings])`	Read logged data for the console's Results page and 3D playback.
`[t, y] = spot.data.samples(ts)`	Time and data of a logged signal as plain columns.
`folder = spot.data.saveSimulation(s, root)`	Keep the last simulation in "Saved Data", next to the experiments.
`r = spot.data.series(ds, a)`	One signal of a dataset, ready to plot in the console.
`s = spot.data.summary(ds, id)`	What the console shows about a loaded dataset (no data).
`fig = spot.data.toFigure(panels, lookup[, opts])`	The Results page's plots as a MATLAB figure in the lab's publication style.

Call	Purpose
`r = spot.data.trajectory(ds[, maxRate])`	A dataset's motion for the console's 3D view.

spot.experiment – Experiments on the Platforms

Call	Purpose
`out = spot.experiment.build(s, root)`	Build the selected diagram for the platforms, on the lab console (TX2).
`b = spot.experiment.buildBoard(root[, s])`	Where experiments are built: the lab console (Jetson TX2).
`info = spot.experiment.connect(root[, password])`	Connect MATLAB to the build board (the TX2), as jetson() does for a platform.
`files = spot.experiment.decodeData(folder, s, root)`	Turn the experiment data files (ExpLog*) into MATLAB timeseries.
`p = spot.experiment.plan(s, root)`	What spot.experiment.build would do, without building anything.

spot.hardware – Code the Spacecraft Program Uses to Talk to Its Hardware

Call	Purpose
MATLAB System block `spot.hardware.ArmTorque (torque) → (q, dq, torque, status)`	The arm's three Dynamixel servos in current control: reads their encoders and torques before the controllers, sends the joint torques at the end of the step (the Dynamixel Arm (Torque) block).
MATLAB System block `spot.hardware.CustomDataLogger (data, size) → (none)`	Appends each step's logged values to the platform's ExpLog file (the diagram's Data Logger block).
MATLAB System block `spot.hardware.Debugging (value) → (none)`	Prints its input on the platform's console while an experiment runs.
`spot.hardware.debugMessages(whoAmI, isSim, t, red, black, blue[, labConsoleIP])`	Each platform's debug message, for the SPOT Console's Messages panel.
`[t, state] = spot.hardware.experimentTime(packet, age)`	The experiment's time, the same on every platform, from the PhaseSpace stream.
MATLAB System block `spot.hardware.IdentifyPlatform () → platformId`	Which platform the program runs on (1 RED, 2 BLACK, 3 BLUE), from its hostname (the WhoAmI block).
`s = spot.hardware.ipText(ip)`	An IPv4 address as text for the C code: '192.168.1.116' from [192 168 1 116].
`spot.hardware.liveStream(label, t, u, whoAmI, isSim[, labConsoleIP])`	Send a signal to the SPOT Console's Live Plot, every step (the Live Stream block).
`packet = spot.hardware.packCommand(duty, pucks)`	The 37-byte command packet for a platform's Teensy (8 thruster duty cycles and the pucks).
`[packet, newPackets, age] = spot.hardware.phasespaceLatest(port, senderIP)`	The newest PhaseSpace packet every model step, for the diagram's "Receive PhaseSpace Data" block.
`label = spot.hardware.streamName(block, name)`	The name a Live Stream block shows in the SPOT Console's Live Plot.
`name = spot.hardware.Udp(~)`	Connects csrc/udp/spot_udp.c to the generated code (used by the UDP functions here).
`[data, newPackets, age] = spot.hardware.udpReceive(port, senderIP, n, type)`	Newest UDP packet every model step (replaces the UDP Receive block).

Call	Purpose
`spot.hardware.udpSend(ip, port, data)`	Send a signal as one UDP packet every model step (replaces the UDP Send block).

spot.massprops – Mass Properties

Call	Purpose
`I = spot.massprops.bifilarInertia(mass, T, L, D)`	Moment of inertia from a bifilar (two-string) pendulum test.
`r = spot.massprops.centerOfGravity(scales)`	Mass, centre of gravity, thruster offsets and LED positions from scale readings.
`r = spot.massprops.compute(mass)`	Mass properties of RED, BLACK and BLUE from the settings' mass section.
`text = spot.massprops.trackerPositions(props)`	PhaseSpace marker calibration text from the computed mass properties.
`file = spot.massprops.writeTrackerPositions(props, root)`	Write resources/tracker_positions_phasespace.txt for the PhaseSpace server.

spot.params – Settings

Call	Purpose
`d = spot.params.apply(s[, root])`	Put the settings into the base workspace, as the Simulink diagrams expect them.
`s = spot.params.defaults()`	Default SPOT settings.
`s = spot.params.fromLegacyState(file)`	Convert a GUI state file saved by the old SPOT GUI (Save GUI State) to settings.
`ip = spot.params.ipv4(address)`	An IPv4 address as four numbers: [192 168 1 116] from '192.168.1.116'.
`s = spot.params.load(file)`	Read settings from a JSON file, filling anything missing with the defaults.
`c = spot.params.loadCalibration(root)`	The project's calibration of the lab (resources/spot_calibration.json).
`s = spot.params.merge(base, changes)`	Copy the fields of CHANGES over BASE, recursing into structs.
`n = spot.params.platformSelection(p)`	The diagrams' platformSelection code for a set of platforms.
`[s, backup] = spot.params.reset(current, root, settingsFile, backupFolder)`	The template's default settings, keeping the diagram, after backing up the settings file.
`spot.params.save(s, file)`	Write settings to a JSON file (readable, and diff-friendly in version control).
`t = spot.params.timing(s)`	Phase durations, end times and total time from the settings.
`s = spot.params.update(old, changes, root)`	New settings from the console: the defaults overlaid with what the page sent, re-reading the solver for a new diagram.

spot.plotting – Drawing

Call	Purpose
`P = spot.plotting.armPosition(base, q, arm)`	Joint positions of the robotic arm on RED, in the table frame.
`links = spot.plotting.drawArm(joints)`	Outlines of the arm's three links, for publication figures.
`pts = spot.plotting.drawSpacecraft(x, y, theta, id)`	Outline of a SPOT platform on the table, for publication figures.
`[name, color] = spot.plotting.platformStyle(key)`	What a platform is called and its colour in figures.
`L = spot.plotting.signalLabel(name)`	Readable name and unit of a logged signal, for figure labels.

spot.sim – Simulations

Call	Purpose
`out = spot.sim.debugLog(platform, t, codes)`	The platforms' debug messages recorded during a simulation (by spot.hardware.debugMessages).
`d = spot.sim.errorDetails(err[, root])`	An error's root cause, the blocks involved and the full report, as plain text.
`names = spot.sim.listDiagrams(root)`	Simulink diagrams (.slx) in the project folder.
`out = spot.sim.liveData(name, t, v)`	The Live Stream blocks' signals recorded during a simulation (by spot.hardware.liveStream).
`model = spot.sim.loadDiagram(diagram, root)`	Load a diagram from the project folder (without opening its window).
`name = spot.sim.modelName(diagram)`	Simulink model name of a diagram file ('Template.slx' -> 'Template').
`solver = spot.sim.readSolver(diagram, root, solver)`	Copy a diagram's own solver settings into the settings' solver section.
`r = spot.sim.results(dataClass, s[, d, maxRate])`	Trajectory of a simulation for the SPOT Console's 3D view.
`out = spot.sim.run(s, root[, resultFile])`	Simulate the selected diagram with the given settings.
`block = spot.sim.showBlock(diagram, root, block)`	Open a diagram at a block and highlight it (e.g. the block an error is in).
`obj = spot.sim.SimulationFailed(original, output)`	The error spot.sim.run throws when Simulink stops a simulation with an error.
`a = spot.sim.tableSlope(position, velocity, slope, damping)`	The table's pull on a floating platform: its slope and the air bearings' drag.

spot.worker – The MATLAB Worker That Runs the Console's Commands

Call	Purpose
`w = spot.worker.start(port, token, root)`	Connect a MATLAB worker to the SPOT Console's web server.
`spot.worker.stop()`	Disconnect the MATLAB worker from the SPOT Console (if one is running).
`w = spot.worker.Worker(root); result = w.dispatch(cmd, args); w = spot.worker.Worker.instance()`	Runs the console's MATLAB commands in this session (the commands are listed in Appendix B).

spot.workspace – The Console's Workspace Panel and Command Window

Call	Purpose
`r = spot.workspace.evaluate(command)`	Run a command in the base workspace and capture what it prints.
`r = spot.workspace.inspect(expr)`	Details of a workspace variable (or a field of one) for the Workspace panel.
`vars = spot.workspace.list()`	The base workspace's variables, as MATLAB's Workspace browser shows them.
`t = spot.workspace.preview(x)`	Short text for a value, like the Value column of MATLAB's Workspace browser.

B The Console's Commands and the Non-MATLAB Parts

The local console asks MATLAB for everything through `spot.worker.Worker.dispatch(cmd, args)`; each command calls the functions above.

Commands of the local console (`dispatch`).

Command	Purpose
`ping, info`	Check that MATLAB answers; its release and toolboxes.
`settings.get, settings.set`	Read or replace the settings (spot.params.update), then apply them.
`settings.reset`	Back to the defaults (spot.params.reset).
`settings.open, settings.saveAs`	Load or save settings through MATLAB's file dialogs.
`diagram.open`	Open the diagram in Simulink.
`sim.run, sim.stop`	Run the simulation (spot.sim.run), or stop it.
`experiment.plan, .connect, .build`	spot.experiment.plan, connect and build.
`experiment.decode`	Convert collected ExpLog files (spot.experiment.decodeData).
`data.list, data.load, data.unload`	Saved Data, and the datasets loaded for the Results tab (spot.data.listSaved, load).
`data.series, data.trajectory`	A signal to plot, or a dataset's motion for the 3D view (spot.data.series, trajectory).
`data.saveSimulation, data.figure`	Save the last simulation; draw or export the plots (spot.data.toFigure, exportFigure).
`massprops.writeTracker`	Write the PhaseSpace marker calibration (spot.massprops.writeTrackerPositions).
`workspace.list, .inspect, .eval`	The Workspace panel and the command window (spot.workspace.*).

The non-MATLAB parts.

File	Purpose
`+spot/+console/webapp/app.py`	Starts the console's web server: `-local` for the local console (SPOT does this), otherwise the hardware console (the TX2's service).
`webapp/spot_console/common.py`	Paths, the API error, checking values from requests.
`webapp/spot_console/events.py`	The console's own settings, the event stream to every open page, running commands.
`webapp/spot_console/phasespace.py`	Compiling and running StreamData, decoding its packets for the 3D view, marker calibration.

The non-MATLAB parts.

File	Purpose
webapp/spot_console/hardware.py	Teensy packets and the SSH links to the platforms: pucks, thrusters, thruster checks.
webapp/spot_console/control.py	The diagram's default LQR law and thruster mixer in Python (spot.control.lqrCommand and thrusterDutyCycles, ported line by line), for Interactive Mode.
webapp/spot_console/interactive.py	Interactive Mode: flies one or more platforms to their goals from the PhaseSpace stream, each with its own loop, controller process and safety stops.
webapp/spot_console/experiment.py	Deploying, starting, stopping and collecting experiments on the platforms.
webapp/spot_console/local.py	The local console: the MATLAB link, simulation and build runs, the link to the lab console.
webapp/spot_console/files.py	The folder browser.
webapp/spot_console/design.py	The Spacecraft Designer's design (docking cone and solar panels on each platform): checked and kept in a JSON file.
webapp/spot_console/calibration.py	The Calibration tab: the calibration file (the lab's, and each project's copy), and the calibrations, which drive a platform (the table's slope from drifts).
webapp/spot_console/calfit.py	The calibrations' fits (linear least squares on the measured positions): the table's height surface and drag, each thruster's thrust.
webapp/spot_console/server.py	The web API and start-up.
webapp/static/	The page: app.js (common), sim.js, experiment.js, results.js, hardware.js, interactive.js (Interactive Mode), designer.js (Spacecraft Designer), calibration.js and calviz.js (Calibration), viewer.js (3D), cad.js (Use CAD), data.js.
+spot/+console/phasespace/StreamData.cc	Reads PhaseSpace and streams positions to the platforms (UDP 31534); built with its Makefile or build_linux.sh.
+spot/+console/deploy/deploy_tx2.sh	Installs the console on the TX2 and (re)starts its service (spot-phasespace-console.service).
+spot/+console/deploy/ setup_hardware_link.sh	Installs the platform agent on RED/BLACK/BLUE and gives the TX2 access to it.
+spot/+console/deploy/spot_agent.sh	The only thing the TX2 may run on a platform: status, stream, exp-receive, exp-start, exp-stop, exp-status, exp-data, exp-clean.
+spot/+hardware/csrc/	C/C++ behind the hardware blocks (arm/arm_torque.cpp for the arm, WhoAmI, Data Logger, Debugging, PhaseSpace receiver).
+spot/+hardware/arm/	setup_arm_serial.sh: on RED, the U2D2 adapter's 1 ms latency timer and permissions (a udev rule).
+spot/+hardware/teensy/	The Teensy firmware (teensy_pwm_control.ino) and setup_teensy_serial.sh.